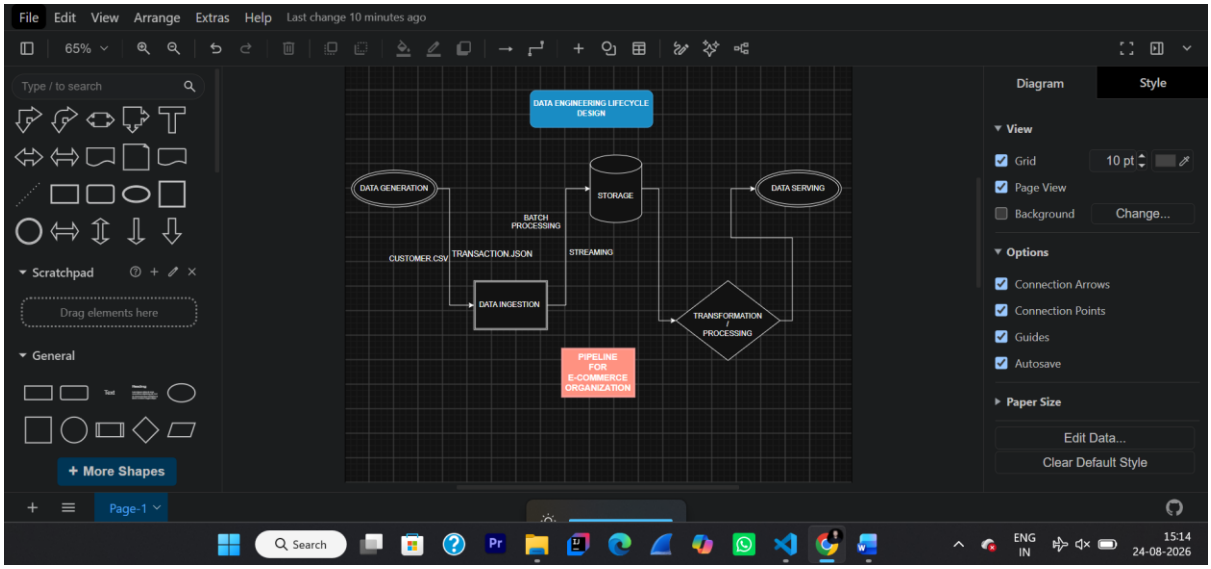
CONTINUOUS INTERNAL PRACTICAL EVALUATION – 1
SUBJECT : DATA ENGINEERING [ITUC301]

Problem Definition :

An e-commerce organization collects data from multiple sources, including:
• Customer and product information stored in a relational database.
• Transaction data received through an API in JSON format.
• Web application user activity logs generated continuously.

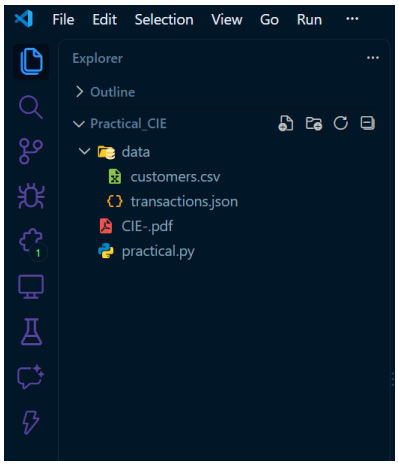
You are required to design and implement a simplified end-to-end data engineering pipeline.

TASK 1 : Data Engineering Lifecycle Design – DRAW.IO



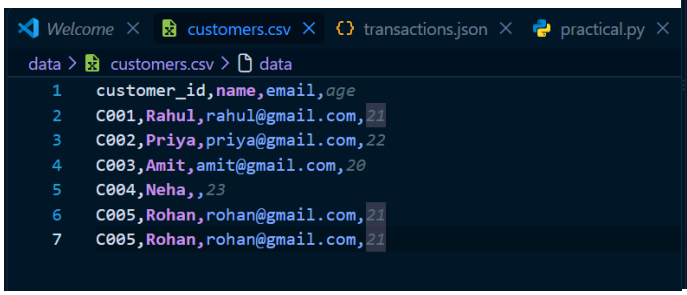
TASK 2 : Data Generation and Profiling

STEP 1 : Created Structure in the folder

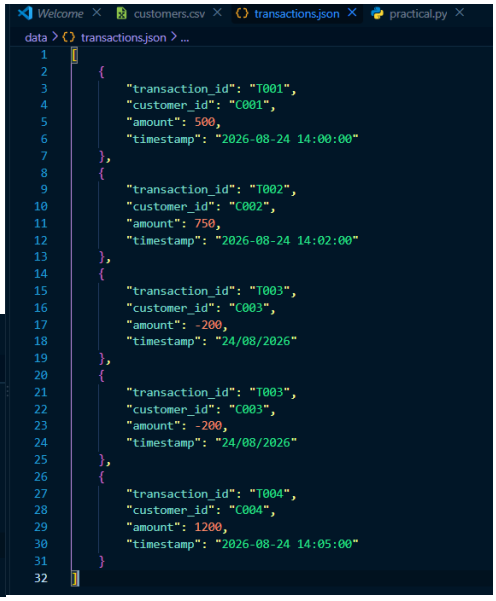


FOLDER_STRUCTURE

STEP 2 : Generated Customer.csv and transaction.json

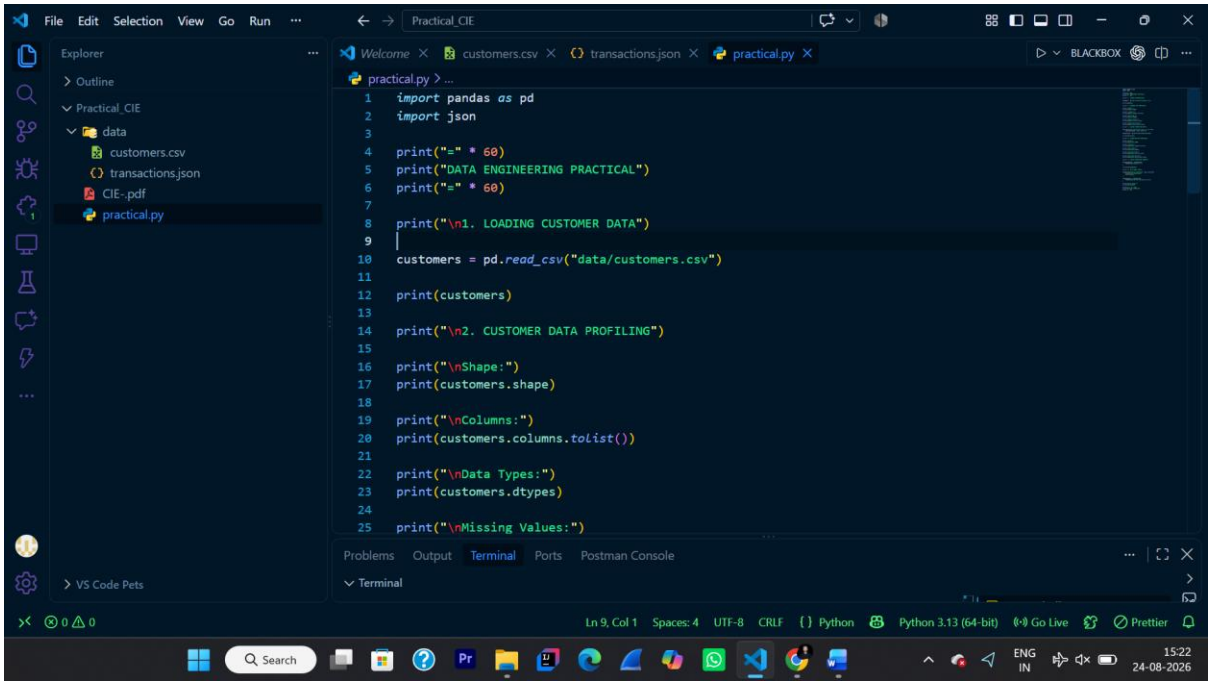


CUSTOMER.CSV



TRANSACTION.JSON

STEP 3 : Created Practical.py

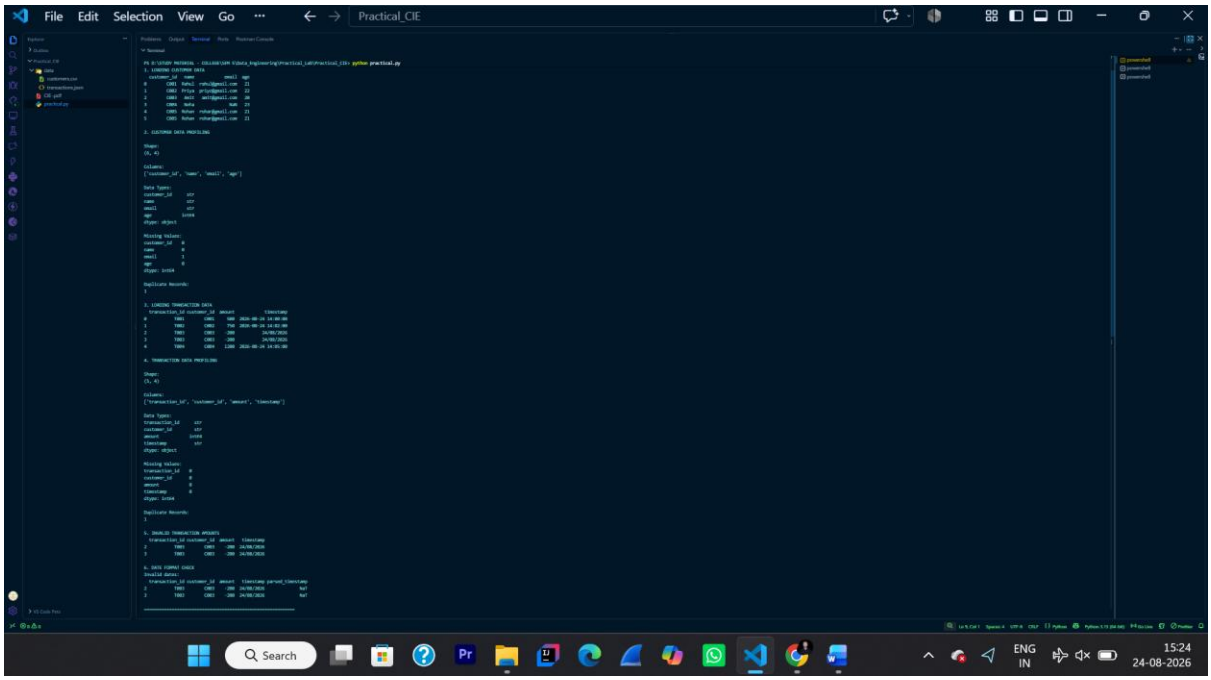


PRACTICAL.PY

STEP 4 : RUN practical.py

Command – python practical.py

OUTPUT :



IDENTIFIED QUALITY ISSUES :

ISSUE 1 : CUSTOMER DATA

Different types of data
Missing Values
Duplicate Records

```
Data Types:
customer_id      str
name             str
email            str
age            int64
dtype: object

Missing Values:
customer_id    0
name           0
email          1
age            0
dtype: int64

Duplicate Records:
1
```

ISSUE 2 : TRANSACTION DATA

Different types of data
Invalid Amount
Invalid dates

```
Data Types:
transaction_id     str
customer_id        str
amount           int64
timestamp          str
dtype: object

Missing Values:
transaction_id    0
customer_id       0
amount            0
timestamp         0
dtype: int64

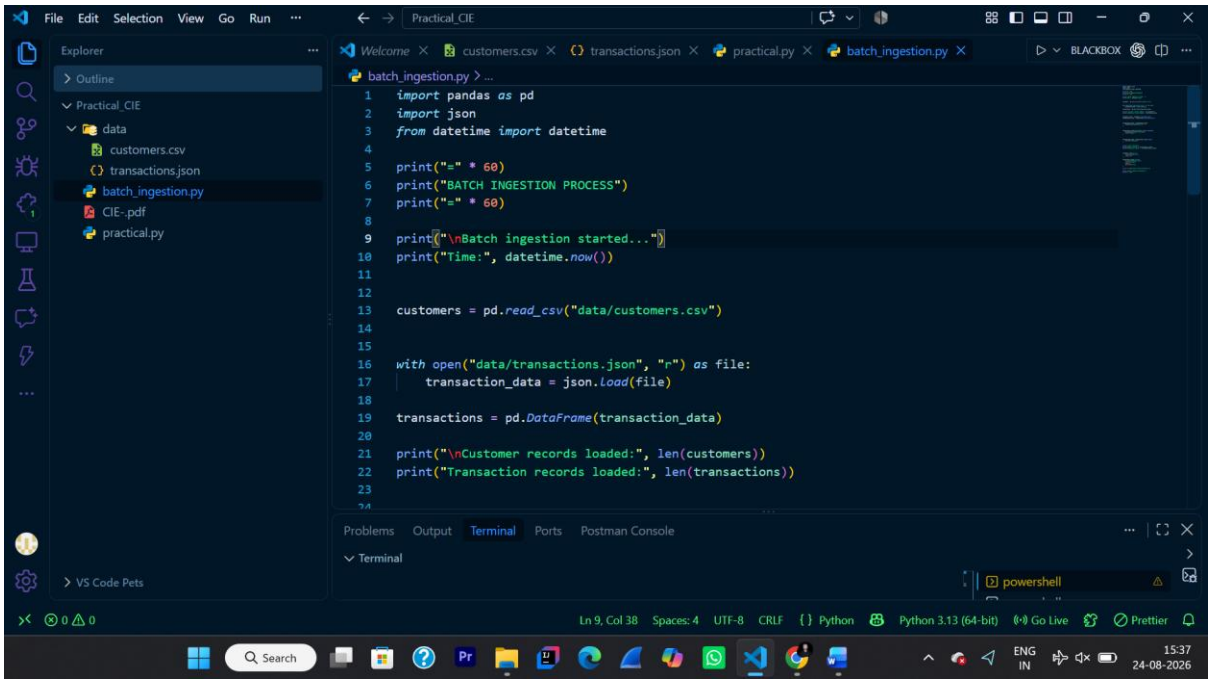
Duplicate Records:
1

5. INVALID TRANSACTION AMOUNTS
  transaction_id customer_id  amount   timestamp
2           T003        C003    -200  24/08/2026
3           T003        C003    -200  24/08/2026

6. DATE FORMAT CHECK
Invalid dates:
  transaction_id customer_id  amount   timestamp parsed_timestamp
2           T003        C003    -200  24/08/2026              NaT
3           T003        C003    -200  24/08/2026              NaT
```

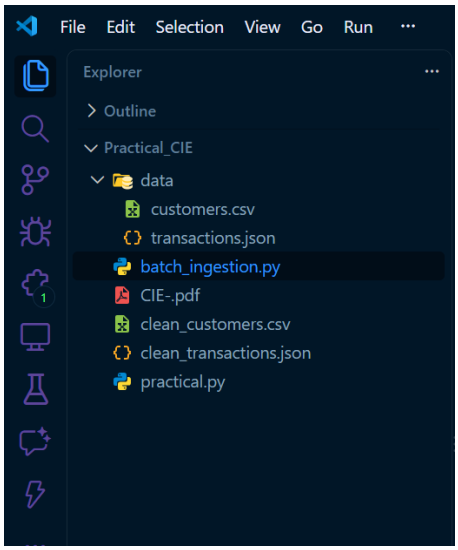
TASK 3 : Batch and Streaming Ingestion

STEP 1 : Created batch_ingestion.py

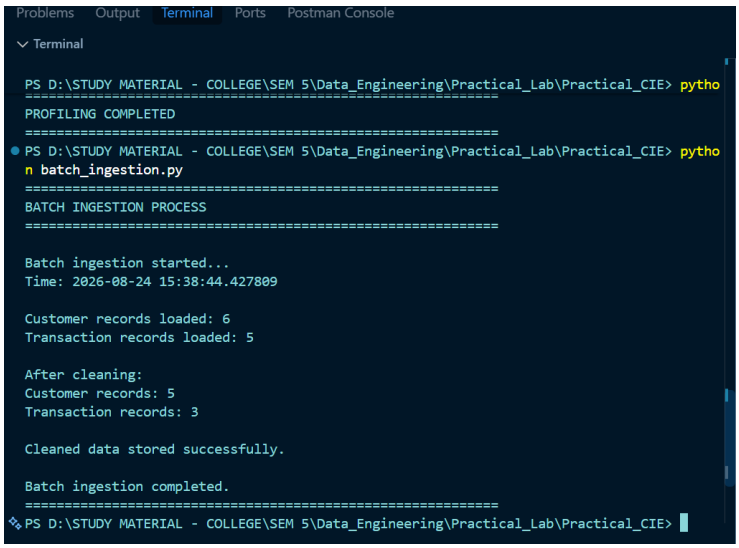


STEP 2 : RUN batch_ingestion.py
Command – python batch_ingestion.py

OUTPUT : clean_customers.csv and clean_transactions.json created

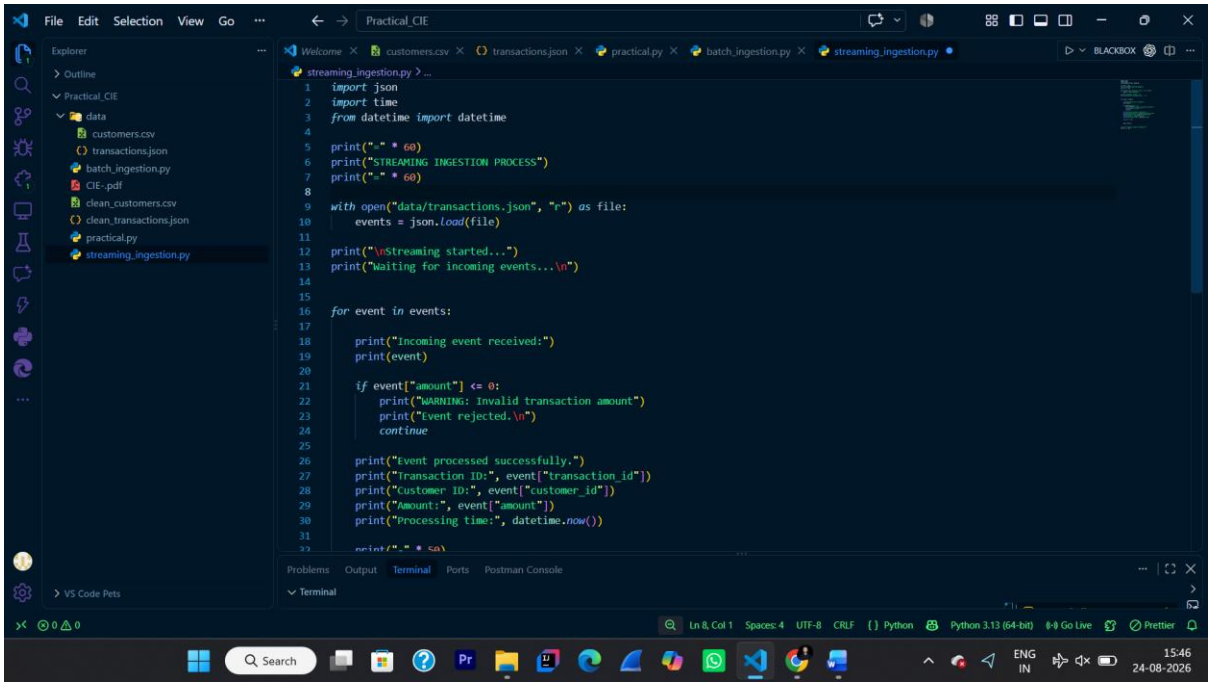


UPDATED FOLDER STRUCTURE



BATCH_INGESTION RUN SUCCESSFULLY AND CLEANED DATA

STEP 3 : Created streaming_ingestion.py



```python
import json
import time
from datetime import datetime

print("-" * 60)
print("STREAMING INGESTION PROCESS")
print("-" * 60)

with open("data/transactions.json", "r") as file:
    events = json.load(file)

print("\nStreaming started...")
print("Waiting for incoming events...\n")


for event in events:

    print("Incoming event received:")
    print(event)

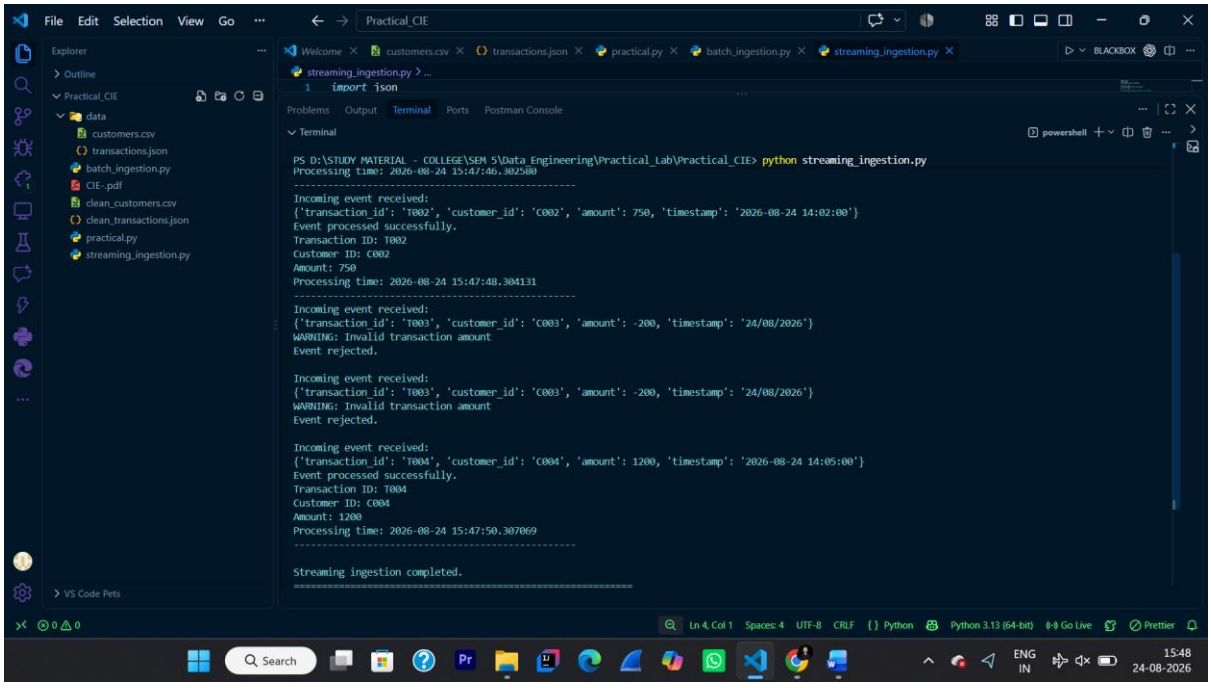
    if event["amount"] <= 0:
        print("WARNING: Invalid transaction amount")
        print("Event rejected.\n")
        continue

    print("Event processed successfully.")
    print("Transaction ID:", event["transaction_id"])
    print("Customer ID:", event["customer_id"])
    print("Amount:", event["amount"])
    print("Processing time:", datetime.now())
```

STEP 4 : RUN streaming_ingestion.py

Command – python streaming_ingestion.py

OUTPUT : Streaming (one by one all event received and processed successfully)



```
PS D:\STUDY MATERIAL - COLLEGE\SEM 5\Data_Engineering\Practical_Lab\Practical_CIE> python streaming_ingestion.py
Processing time: 2026-08-24 15:47:46.302580
------------------------------------------------
Incoming event received:
{'transaction_id': 'T002', 'customer_id': 'C002', 'amount': 750, 'timestamp': '2026-08-24 14:02:00'}
Event processed successfully.
Transaction ID: T002
Customer ID: C002
Amount: 750
Processing time: 2026-08-24 15:47:48.304131
------------------------------------------------
Incoming event received:
{'transaction_id': 'T003', 'customer_id': 'C003', 'amount': -200, 'timestamp': '24/08/2026'}
WARNING: Invalid transaction amount
Event rejected.

Incoming event received:
{'transaction_id': 'T003', 'customer_id': 'C003', 'amount': -200, 'timestamp': '24/08/2026'}
WARNING: Invalid transaction amount
Event rejected.

Incoming event received:
{'transaction_id': 'T004', 'customer_id': 'C004', 'amount': 1200, 'timestamp': '2026-08-24 14:05:00'}
Event processed successfully.
Transaction ID: T004
Customer ID: C004
Amount: 1200
Processing time: 2026-08-24 15:47:50.307069
------------------------------------------------

Streaming ingestion completed.
================================================
```

COMPARISION B/W 2 APPROACHES :

BATCH INGESTION :

➔ Data Processing Latency : Higher
➔ Throughput : Higher
➔ Use case : Suitable for large data

STREAMING INGESTION :

➔ Data Processing Latency : Very Low
➔ Throughput : Good for continuous incoming data
➔ Use case : Useful for real time application

TASK 4 : Result and Analysis

For historical transaction processing, batch ingestion is more suitable
For real-time user activity monitoring, streaming ingestion is more suitable.

Conclusion: Batch processing is better when we have a large amount of stored data, while streaming is better when data needs to be processed continuously and quickly.